AI编码实践：从Vibe Coding到SDD

淘特用户技术团队 大淘宝技术 2025年12月15日 01:47 浙江 333人



本文系统回顾了淘特导购团队在AI编码实践中的演进历程，从初期的代码智能补全到Agent Coding再到引入Rules约束，最终探索SDD（Specification Driven Development，规格驱动开发）——以自然语言规格（spec.md）为唯一真理源，驱动代码、测试、文档自动生成，实现设计先行、可测试性内建与文档永不过期。实践中发现SDD理念先进但落地门槛高、工具链不成熟、历史代码集成难，因此团队当前采用融合策略：**以轻量级技术方案模板为输入 + Rules严格约束 + Agent Coding高效实现 + AI自动汇总架构文档**，形成兼顾规范性、效率与可维护性的AI辅助编程最佳实践。



1.1 业务背景

生成式AI技术的范式突破正驱动智能开发工具进入超线性演进阶段，主流代码生成工具的迭代周期已从季度级压缩至周级，智能体架构创新推动开发效能持续提升。

淘特导购系统承载着商品推荐、会场投放、活动营销等多样化的业务场景，技术团队面临着需求迭代频繁、代码腐化及团队协作度高的问题，如何提升开发效率、保证代码质量、降低维护成本成为我们面临的重要挑战。正是在这样的背景下，我们开始尝试将AI技术融入到日常开发流程中，探索从传统编码到AI辅助编程的转变之路。

1.2 AI编程工具的引入

2024年初，团队开始探索AI编程工具，希望通过AI提升开发效率和代码质量。最初接触的是Aone Copilot（阿里内部AI工具）的代码智能补全功能，后来逐步尝试Agentic Coding、Rules约束、SDD（Specification Driven Development）等多种AI编程模式。本文将详细记录我们的探索历程、实践经验以及对AI编程未来的思考。

02 代码智能补全与单方法改写

2.1 初识AI编程

场景1：代码自动补全

```
// 开发者输入：
public List<ItemCardVO> buildItemCards(List<ContentEntity> entities) {
    List<ItemCardVO> result = new ArrayList<>();
    // AI自动补全以下代码
    for (ContentEntity entity : entities) {
        ItemCardVO itemCard = new ItemCardVO();
        itemCard.setItemId(entity.getItemId());
        itemCard.setItemTitle(entity.getTitle());
        itemCard.setItemImg(entity.getPicUrl());
        result.add(itemCard);
    }
    return result;
}
```

场景2：单方法重构

```
// 原始代码（冗长难读）
public String getDiscountText(Long finalPrice, Long nnPrice) {
    if (finalPrice == null || nnPrice == null) {
        return "";
    }
    if (finalPrice <= nnPrice) {
        return "";
    }
    Long discount = finalPrice - nnPrice;
    if (discount <= 0) {
        return "";
    }
    String discountYuan = String.valueOf(discount / 100.0);
    return discountYuan + "元";
}
// AI重构后（简洁优雅）
public String getDiscountText(Long finalPrice, Long nnPrice) {
    if (finalPrice == null || nnPrice == null || finalPrice <= nnPrice
        return "";
```

```
    }
    Money discount = Money.ofFen(finalPrice).subtract(Money.ofFen(nnPr
    if (discount.getCent() <= 0) {
        return "";
    }
    return String.format("%s元", discount.getYuan());
}
```

2.2 初步收益

效率提升：

- 去年某次商详项目中，代码补全在对象构建、模型转换中减少70-80%的键盘输入。
- 单方法重构速度提升50%。

体验优化：

- 减少了查找API文档的时间
- 避免了拼写错误和语法错误
- 让开发者更专注于业务逻辑

2.3 遇到的问题

然而，这个阶段也暴露出一些问题：

1. 局限于局部优化：只能帮助完成单个方法或代码片段，无法理解整体业务逻辑
2. 缺乏上下文理解：不了解项目的架构规范和代码风格
3. 无法应对复杂需求：对于跨多个类、多个模块的需求无能为力



Agentic Coding的探索与挑战

3.1 Agentic Coding的尝试

带着上面的问题，我们开始尝试Agentic Coding模式，通过编写详细的提示词（Prompt），让AI一次性实现整个功能。

AI工具：Aone Copilot，模型：Qwen3-coder。

典型的Prompt结构：

```
需求：实现NN页面红包模块
背景：
```

```
- 需要展示用户可用的红包列表
- 红包按门槛从小到大排序
实现要求：
1. 创建数据服务类 NnRedPacketDataService，查询用户红包
2. 创建模块VO NnRedPacketVO，包含红包列表、总金额等字段
3. 创建模块构建器 NnRedPacketModuleBuilder，组装数据
技术细节：
- 数据服务需要实现 DataService<List<FundQueryDTO>> 接口
- 数据服务实现类需要依赖FpProvider，并执行红包查询，参数请参考已有实现
- 模块构建器需要继承 BaseModuleBuilder<NnRedPacketVO>
- 使用 @Component 注解标记为Spring Bean
- 遵循项目代码规范
请生成完整的代码。
```

AI生成的代码示例：

```java
// 1. 数据服务
@Component
public class NnRedPacketDataService implements DataService<List<FundQu
    @Autowired
    private FpProvider fpProvider;
    @Override
    public List<FundQueryDTO> execute(InvocationContext context, JSONO
        // 从配置获取红包池ID
        String poolIds = req.getString("nnRedPacketPoolIds");
        List<Long> fundPoolIds = Arrays.stream(poolIds.split(","))
            .map(Long::parseLong)
            .collect(Collectors.toList());
        // 查询用户红包
        return fpProvider.queryUserFundBuyPoolId(
            context,
            fundPoolIds,
            customRuleId,
            securityCode
        );
    }
}
// 2. 模块VO
@Data
@Builder
public class NnRedPacketVO {
    private List<RedPacketItem> redPacketList;
    private String totalAmount;
    private String expandText;
}
// 3. 模块构建器
```

```java
@Component
public class NnRedPacketModuleBuilder extends BaseModuleBuilder<NnRedP
    @Override
    public String getName() {
        return "nnRedPacket";
    }
    @Override
    protected NnRedPacketVO doBuild(InvocationContext context) {
        // 获取红包数据
        List<FundQueryDTO> funds = ContextUtils
            .<List<FundQueryDTO>>getBizResult(
                NnRedPacketDataService.class.getSimpleName(),
                context,
                500
            )
            .orElse(Collections.emptyList());
        if (CollectionUtils.isEmpty(funds)) {
            return null;
        }
        // 构建VO
        return NnRedPacketVO.builder()
            .redPacketList(buildRedPacketList(funds))
            .totalAmount(calculateTotalAmount(funds))
            .build();
    }
    private List<RedPacketItem> buildRedPacketList(List<FundQueryDTO>
        return funds.stream()
            .map(this::toRedPacketItem)
            .collect(Collectors.toList());
    }
    // ... 其他方法
}
```

3.2 显著的效率提升

Agentic Coding实现了开发效率的显著优化，通过提示词驱动的自动代码生成替代了传统手动编写，虽然Prompt设计需要额外时间，但综合效率提升效果明显。

3.3 快速暴露的问题

然而，在后续的需求迭代中，我们发现了严重的问题：

问题1：代码延续性差

现象： 同样的NN业务，第二次让AI实现时，生成的代码风格完全不同

```
// 第一次生成（简洁风格）
private String buildDiscountText(Money discount) {
    return String.format("省%s元", discount.getYuan());
}
// 第二次生成（冗长风格）
private String buildDiscountText(Money discount) {
    BigDecimal yuan = BigDecimal.valueOf(discount.getCent())
        .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    String yuanStr = yuan.stripTrailingZeros().toPlainString();
    return "省" + yuanStr + "元";
}
```

影响： 同一个项目内，类似功能的实现方式五花八门，维护成本高

问题2：代码风格不一致

现象： AI不了解项目的代码规范，导致生成的代码风格和存量代码不一致。

问题3：团队协同性差

现象： 不同开发者写的Prompt差异大，生成的代码质量参差不齐

- 新手写的Prompt过于简单，AI生成的代码质量差
- 老手写的Prompt详细但冗长，难以复用
- 缺乏统一的Prompt模板和最佳实践

3.4 原因分析

这些问题的根本原因在于：**AI缺乏项目特定的上下文和约束**

1. **没有项目规范：** AI不知道项目的代码风格、架构模式、命名规范
2. **没有领域知识：** AI不了解淘特导购业务的特定术语和设计模式
3. **没有历史经验：** 每次都是"零基础"生成代码，无法从历史代码中学习

这让我们意识到，需要给AI建立"项目规范"和"领域知识"。

04 Rules约束 – 建立AI的"项目规范"

4.1 引入Rules文件

我们开始尝试用Rules文件来约束AI的行为，将项目规范、架构模式、领域知识固化下来。

Rules文件体系：

```
.aone_copilot/
├── rules/
│   ├── code-style.aonerule            # 代码风格规范
│   ├── project-structure.aonerule     # 项目结构规范
│   └── features.aonerule               # 功能实现规范
└── tech/
    ├── xx秒杀-技术方案.md       # 具体需求的技术方案
    └── xx红包模块-技术方案.md
```

4.2 Rules文件内容示例

代码风格规范（code-style.aonerule）

```
# 代码风格规范

## Java代码规范
- 类名使用大驼峰命名法（PascalCase）
- 方法名和变量名使用小驼峰命名法（camelCase）
- 常量使用全大写，单词间用下划线分隔（CONSTANT_CASE）

## 空值判断
- 集合判空统一使用：CollectionUtils.isEmpty() 或 isNotEmpty()
- 字符串判空统一使用：StringUtils.isBlank() 或 isNotBlank()
- 对象判空统一使用：Objects.isNull() 或 Objects.nonNull()

## 日志规范
- 使用 LogUtil 工具类记录日志
- 错误日志格式：LogUtil.error("类名, 方法名, 错误描述, 关键参数={}", param, e

## 注解使用
- Service类使用 @Component 注解
- 数据服务实现 DataService<T> 接口
- 模块构建器继承 BaseModuleBuilder<T>
```

项目结构规范

```
# 项目结构规范
## 包结构
com.alibaba.aladdin.app/
├── module/                # 模块构建器
│   ├── nn/              # NN业务模块
│   ├── seckill/         # 秒杀业务模块
```

```
│   └── common/          # 通用模块
├── domain/              # 领域对象
│   ├── module/          # 模块VO（继承ModuleObject）
│   └── [业务名]/        # 业务领域对象（BO、DTO）
├── dataservice/impl/    # 数据服务实现
└── provider/            # 外部服务提供者
## 命名规范
- 数据服务：[业务名]DataService（如 NnRedPacketDataService）
- 模块构建器：[业务名]ModuleBuilder（如 NnFeedsModuleBuilder）
- 模块VO：[业务名]VO（如 NnRedPacketVO）
- 业务BO：[业务名]BO（如 NnRoundFeatureBO）
```

功能实现规范

```
# 功能实现规范
## 数据服务层
- 必须实现 DataService<T> 接口
- 使用 @Component 注解
- execute方法的第一个参数是 InvocationContext
- execute方法的第二个参数是 JSONObject businessReq
示例：
```java
@Component
public class NnRedPacketDataService implements DataService<List<FundQu
    @Override
    public List<FundQueryDTO> execute(InvocationContext context, JSONO
        // 实现逻辑
    }
}
```
模块构建器
- 必须继承 BaseModuleBuilder
- 使用 @Component 注解
- 实现 getName()、doBuild()、bottomTransform() 三个方法
- 通过 ContextUtils.getBizResult() 获取数据服务结果
示例：
```
@Component
public class NnRedPacketModuleBuilder extends BaseModuleBuilder<NnRedP
    @Override
    public String getName() {
        return "nnRedPacket";
    }
    @Override
    protected NnRedPacketVO doBuild(InvocationContext context) {
```
```

```
        List<FundQueryDTO> funds = ContextUtils
            .<List<FundQueryDTO>>getBizResult(
                NnRedPacketDataService.class.getSimpleName(),
                context,
                500
            )
            .orElse(Collections.emptyList());
        // 构建逻辑
    }
}
```

4.3 技术方案模板

除了Rules文件，我们还为每个需求创建技术方案文档，明确定义需要生成的代码：

技术方案示例（NN红包模块-技术方案.md）：

```
## 业务定义
NN红包模块用于展示用户在NN业务场景下可用的红包列表。
## 业务领域对象
无（复用 FundQueryDTO）
## 模块领域对象
| 对象含义 | 实现方案 | 属性及类型 |
|---------|---------|-----------|
| NN红包模块VO | 新增 | 1. redPacketList：List<RedPacketItem> - 红包列表<b
## 数据服务层
| 数据服务定义 | 实现方案 | execute |
|------------|---------|---------|
| NN红包查询服务 | 新增 | 1. 从配置获取红包池ID列表<br>2. 调用FpProvider查询用
## 模块构建器
| 模块构建器定义 | 实现方案 | doBuild逻辑 |
|--------------|---------|-------------|
| NN红包模块构建器 | 新增 | 1. 获取红包数据<br>2. 过滤门槛>20元的红包<br>3. 按
```

4.4 显著改善的效果

引入Rules文件后，我们看到了明显的改善：

代码一致性：

- 所有生成的代码都遵循统一的命名规范
- 项目结构清晰，模块划分明确

- 代码风格保持一致

开发效率：

- 技术方案填写时间从2小时降低到20分钟
- 代码实现时间从1天降低到2小时（需要人工收尾）

团队协作：

- 技术方案成为团队共同语言
- Code Review效率提升50%
- 新人上手时间从1周降低到2天

4.5 依然存在的问题

虽然Rules带来了显著改善，但仍存在一些问题：

1. **需求理解不够深入**：AI仍然是基于技术方案"翻译"成代码，对业务语义理解有限
2. **测试质量参差不齐**：虽然能生成单测，但测试用例的通过率和覆盖度仍需人工把关
3. **文档滞后**：代码变更后，文档更新容易遗漏
4. **依赖关系管理**：对于复杂的模块依赖关系，AI处理不够优雅

这些问题让我们思考：能否找到一种方式，让AI能更加规范和延续的coding?

05 SDD探索 – 规格驱动开发

5.1 SDD的引入

近期，我们开始初步尝试SDD（Specification Driven Development，规格驱动开发），使用了Spec Kit工具链。

SDD的核心理念：

规格是唯一真理源（Single Source of Truth）

- 所有的代码、测试、文档都从规格生成
- 规格即文档，文档永不过期

设计先于实现

- 先用自然语言描述"做什么"（规格）
- 再让AI生成"怎么做"（代码）

可测试性内建

- 规格中明确定义测试用例
- 自动生成完整的单元测试

5.2 Speckit执行流程

5.2.1 环境准备

我们主要使用了两种工具：

1. iflow + qwen3 coder plus + spec kit
2. qwen + qwen3 coder plus + spec kit

文件体系：

```
├── .specify/
│   ├── memory/
│   │   └── constitution.md
│   ├── scripts/
│   └── templates/
├── specs/
│   └── 001-nn-redpacket-module/
│       ├── checklists/
│       │   └── requirements.md
│       ├── contracts/
│       │   └── api-contract.md
│       ├── data-model.md
│       ├── plan.md
│       ├── quickstart.md
│       ├── research.md
│       └── spec.md
└── req/
    └── nn-redpacket.md
```

5.2.2 speckit.constitution—制定整个项目的原则

这一步会生成项目全局的宪章，constitution.md

以下是部分节选：

```
## 核心原则

### I. 模块化服务架构
所有服务必须遵循模块化设计原则，具有明确的关注点分离和定义良好的接口。每个模块应具有卓
### II. 阿里巴巴开发标准
所有代码必须遵循阿里巴巴Java开发指南（基于阿里巴巴Java编码规范）。这包括命名约定、异
### III. 质量保证实践
全面测试是强制性的：对所有业务逻辑进行单元测试，对服务交互进行集成测试，对API兼容性进
### IV. 模块设计原则
遵循单一职责原则，每个模块都有一个明确的目的。模块必须以松耦合和高内聚的方式设计，遵循
### V. 项目架构设计原则
本项目采用分层架构设计，通过模块化组织代码，支持淘特投放业务的各种场景需求。架构层次包
1. **接入层**：处理请求接入和协议转换
```

```
2. **解决方案层**：业务解决方案的统一入口
3. **子解决方案层**：细粒度的业务处理能力
4. **模块构建层**：按业务功能划分的模块构建器
5. **数据服务层**：负责各种业务数据的获取、处理和封装
6. **外部服务层**：负责调用外部服务并进行模型转换
7. **领域模型层**：定义核心业务对象和数据传输对象
8. **基础设施层**：包含基础组件和框架封装
9. **通用模块层**：公共组件和工具类
### VI. 依赖管理
遵循最小依赖原则：每个模块应只拥有其实际需要的依赖项。避免模块之间的循环依赖。使用依
### VII. 代码风格一致性
在整个项目中保持一致的代码风格，使用标准化的格式化规则。所有代码在合并前必须通过静态分
### VIII. Speckit中文本地化
所有speckit相关文件、文档和配置都应使用中文，以支持本地开发团队。`.specify/`和`sp
## 安全和合规要求
所有代码必须符合阿里巴巴的安全标准，并在部署前进行强制性安全审查。必须为所有暴露的端点
## 开发工作流程
1. 所有代码更改必须遵循标准的阿里巴巴开发工作流程：功能分支、代码审查、自动化测试和C
2. 所有操作不要创建新分支，而是在当前分支下进行
3. 代码生成必须遵循code-generation-prompt.aonerule文件
## 治理
本宪法凌驾于所有其他开发实践之上，必须在存储库中的所有工作中遵循。对本宪法的任何修改者
```

5.2.3 speckit.specify—编写规格说明

这一步会新建一个分支，创建spec.md、requirements.md等文件，这里反复修改了多次，主要还是后续的执行不理想的返工。

NN红包模块规格说明（spec.md）：

```
# NN红包模块规格说明
## 功能概述
NN红包模块用于在NN频道页面展示用户可用的红包列表，帮助用户了解可以使用的优惠。
## 功能需求
### FR-1: 红包数据获取
**描述：** 系统应该能够查询用户在当前NN业务场景下可用的红包
**前置条件：**
- 用户已登录
- 配置了红包池ID（fundPoolIds）
- 配置了规则id（customRuleId）
- 配置了securityCode
**输入：**
- userId：用户ID
- fundPoolIds：红包池ID列表
- customRuleId：自定义规则ID
```

- securityCode：安全码
处理逻辑：
1. 调用FpProvider.queryUserFundBuyPoolId()查询红包
2. 过滤条件：
   - 红包状态（payStatus）= 2（可使用）
   - 红包未过期（当前时间在startTime和endTime之间）
   - 红包门槛 <= 配置的amountThreshold（默认20元）
输出：
- 返回符合条件的红包列表
异常处理：
- 如果FpProvider调用失败，返回空列表
- 如果用户未登录，返回空列表
...
...
...
处理逻辑：
1. 如果红包列表为空，不展示模块（返回null）
2. 构建NnRedPacketVO：
   - redPacketList：转换每个红包为RedPacketItem
   - totalAmount：计算所有红包金额总和
   - expandText：从配置获取展开文案
输出： NnRedPacketVO
非功能需求
NFR-1：性能要求
- 红包查询超时时间：500ms
- 如果超时，返回空列表，不影响页面其他模块
NFR-2：可扩展性
- 支持配置不同的红包池ID
- 支持配置不同的门槛限制
测试用例
TC-1：正常流程 - 有可用红包
前置条件：
- 用户ID：123456
- 配置的红包池ID：[1001, 1002]
- FpProvider返回2个可用红包：
  - 红包1：金额5元，门槛15元
  - 红包2：金额10元，门槛25元
执行步骤：
1. 调用 doBuild(context)
预期结果：
- 返回的VO不为null
- redPacketList包含2个红包
- totalAmount = "15.00"（5 + 10）
- 红包按门槛排序（红包1在前）
TC-2：边界条件 - 红包门槛超限
前置条件：

```
- FpProvider返回1个红包，门槛25元
- 配置的amountThreshold = 20元（2000分）
**预期结果：**
- 返回null（因为门槛超限，红包被过滤）
...
...
```

5.2.4 speckit.plan—制定实施计划

这一步会生成data-model.md、api.md等几个文件，最重要的是plan.md

5.2.5 speckit.tasks—任务分解

生成task.md

5.2.6 speckit.implement—实现代码

数据服务实现

```java
@Component
public class NnRedPacketDataService implements DataService<List<FundQu
    @Autowired
    private FpProvider fpProvider;
    @Override
    public List<FundQueryDTO> execute(InvocationContext context, JSONO
        long userId = context.getUserId();
        if (userId <= 0) {
            return Collections.emptyList();
        }
        // 获取配置
        String poolIdsStr = businessReq.getString("nnRedPacketPoolIds"
        List<Long> fundPoolIds = parsePoolIds(poolIdsStr);
        Long customRuleId = businessReq.getLong("customRuleId");
        String securityCode = businessReq.getString("securityCode");
        // 查询红包
        List<FundQueryDTO> funds = fpProvider.queryUserFundBuyPoolId(
            context, fundPoolIds, customRuleId, securityCode
        );
        if (CollectionUtils.isEmpty(funds)) {
            return Collections.emptyList();
        }
        // 过滤可用红包（按规格FR-1的过滤条件）
        Long amountThreshold = businessReq.getLong("amountThreshold",
        return funds.stream()
```

```
            .filter(fund -> fund.getPayStatus() == 2)  // 可使用状态
            .filter(fund -> isNotExpired(fund))          // 未过期
            .filter(fund -> fund.getAmountThreshold() <= amountThresho
            .collect(Collectors.toList());
    }
    private boolean isNotExpired(FundQueryDTO fund) {
        long now = System.currentTimeMillis();
        return fund.getStartTime() <= now && now <= fund.getEndTime();
    }
    // ... 其他方法
}
```

模块构建器实现

```
@Component
public class NnRedPacketModuleBuilder extends BaseModuleBuilder<NnRedP
    @Override
    public String getName() {
        return "nnRedPacket";
    }
    @Override
    protected NnRedPacketVO doBuild(InvocationContext context) {
        // 获取红包数据（FR-1）
        List<FundQueryDTO> funds = ContextUtils
            .<List<FundQueryDTO>>getBizResult(
                NnRedPacketDataService.class.getSimpleName(),
                context,
                500
            )
            .orElse(Collections.emptyList());
        // 如果无可用红包，不展示模块（FR-3）
        if (CollectionUtils.isEmpty(funds)) {
            return null;
        }
        // 红包排序（FR-2）
        List<FundQueryDTO> sortedFunds = sortFunds(funds, context);
        // 构建VO（FR-3）
        return NnRedPacketVO.builder()
            .redPacketList(buildRedPacketList(sortedFunds))
            .totalAmount(calculateTotalAmount(sortedFunds))
            .expandText(getExpandText(context))
            .build();
    }
    /**
     * 红包排序：NN专属红包优先，其次按门槛从小到大
     * 对应规格 FR-2
```

```
     */
    private List<FundQueryDTO> sortFunds(List<FundQueryDTO> funds, Inv
        JSONObject extraParams = getExtraParams(context);
        String nnPoolIdsStr = extraParams.getString("nnRedPacketPoolId
        Set<Long> nnPoolIds = parsePoolIds(nnPoolIdsStr).stream()
            .collect(Collectors.toSet());
        return funds.stream()
            .sorted(Comparator
                .comparing((FundQueryDTO fund) ->
                    nnPoolIds.contains(fund.getFromFundId()) ? 0 : 1)
                .thenComparing(FundQueryDTO::getAmountThreshold))
            .collect(Collectors.toList());
    }
    // ... 其他方法
}
```

完整的单元测试

```
@RunWith(MockitoJUnitRunner.class)
public class NnRedPacketModuleBuilderTest {
    @Mock
    private InvocationContext context;
    @Mock
    private NnRedPacketDataService dataService;
    @InjectMocks
    private NnRedPacketModuleBuilder builder;
    /**
     * 测试用例 TC-1: 正常流程 - 有可用红包
     */
    @Test
    public void testBuild_WithAvailableFunds_ShouldReturnVO() {
        // 准备数据
        List<FundQueryDTO> mockFunds = Arrays.asList(
            createFund(1L, 500L, 1500L),   // 5元，门槛15元
            createFund(2L, 1000L, 2500L)   // 10元，门槛25元
        );
        // Mock
        when(dataService.execute(any(), any())).thenReturn(mockFunds);
        mockContext(context, "nnRedPacketPoolIds", "1,2");
        mockBizResult(context, NnRedPacketDataService.class.getSimpleN
        // 执行
        NnRedPacketVO result = builder.doBuild(context);
        // 验证（按规格预期结果）
        assertNotNull(result);
        assertEquals(2, result.getRedPacketList().size());
        assertEquals("15.00", result.getTotalAmount());
```

```
        // 验证排序：门槛低的在前
        assertEquals(1500L, result.getRedPacketList().get(0).getAmount
    }
    /**
     * 测试用例 TC-2: 边界条件 - 红包门槛超限
     */
    @Test
    public void testBuild_ThresholdExceeded_ShouldReturnNull() {
        // 准备数据：门槛25元，超过配置的20元
        List<FundQueryDTO> mockFunds = Arrays.asList(
            createFund(1L, 500L, 2500L)
        );
        // Mock（配置amountThreshold=2000，即20元）
        JSONObject businessReq = new JSONObject();
        businessReq.put("amountThreshold", 2000L);
        when(dataService.execute(any(), eq(businessReq))).thenReturn(C
        mockBizResult(context, NnRedPacketDataService.class.getSimpleN
        // 执行
        NnRedPacketVO result = builder.doBuild(context);
        // 验证（按规格预期结果）
        assertNull(result);
    }
    /**
     * 测试用例 TC-3: 异常场景 - FpProvider返回null
     */
    @Test
    public void testBuild_ProviderReturnsNull_ShouldReturnNull() {
        // Mock
        when(dataService.execute(any(), any())).thenReturn(null);
        mockBizResult(context, NnRedPacketDataService.class.getSimpleN
        // 执行
        NnRedPacketVO result = builder.doBuild(context);
        // 验证（按规格预期结果）
        assertNull(result);
    }
    /**
     * 测试用例 TC-4: 异常场景 - 用户未登录
     */
    @Test
    public void testBuild_UserNotLogin_ShouldReturnNull() {
        // Mock：userId = 0
        when(context.getUserId()).thenReturn(0L);
        when(dataService.execute(any(), any())).thenReturn(Collections
        mockBizResult(context, NnRedPacketDataService.class.getSimpleN
        // 执行
        NnRedPacketVO result = builder.doBuild(context);
```

```
75          // 验证（按规格预期结果）
76          assertNull(result);
77      }
78      // 辅助方法
79      private FundQueryDTO createFund(Long id, Long amount, Long thresho
80          FundQueryDTO fund = new FundQueryDTO();
81          fund.setFundId(id);
82          fund.setAvailableAmount(amount);
83          fund.setAmountThreshold(threshold);
84          fund.setPayStatus(2);  // 可使用
85          fund.setStartTime(System.currentTimeMillis() - 3600000);  // 1
86          fund.setEndTime(System.currentTimeMillis() + 3600000);    // 1
87          return fund;
88      }
89  }
```

5.3 SDD带来的改进

5.3.1 一致性显著提升

代码层面：

- 所有代码都严格遵循规格说明，消除了理解偏差
- 不同开发者实现相同规格，代码风格完全一致
- 代码变更时，必须先更新规格，保证文档与代码同步

业务层面：

- 产品、开发、测试对需求的理解高度一致
- 减少了需求理解偏差导致的返工

5.3.2 可测试性大幅提升

测试覆盖：

- 自动生成的测试用例覆盖了所有正常和异常流程
- 测试用例与规格说明一一对应，确保完整性
- 边界条件和异常场景都有明确的测试用例

测试质量：

- Mock方式规范统一，符合项目最佳实践
- 断言准确全面，不会遗漏关键验证点
- 测试代码可读性好，易于维护

5.3.3 可维护性显著改善

文档永不过期：

- 规格说明就是最准确的文档

- 任何变更都先更新规格，再同步代码
- 新人通过阅读规格说明就能快速理解功能

变更影响分析：

- 修改规格时，清晰知道影响哪些代码模块
- 依赖关系在规格中明确定义
- 重构时可以基于规格验证正确性

代码可读性：

- 代码结构清晰，层次分明
- 注释完整准确，与规格保持一致
- 命名规范统一，易于理解

5.3.4 团队协作效率提升

- 新人通过阅读规格说明快速上手
- 跨团队协作时，规格成为统一语言
- 历史需求回溯更容易，规格即完整记录

5.4 SDD的问题与挑战

虽然SDD带来了价值，但在实践中也遇到了一些明显的问题：

问题1：规格编写门槛高

现象： 编写高质量的规格说明需要较强的抽象能力和文档编写能力

- 新手往往写不好规格，过于技术化或过于模糊
- 规格模板虽然有，但如何填写仍需要经验
- 不合格的规格对后面的代码实现影响

影响： 对于简单需求，写规格的时间甚至超过直接写代码

问题2：Spec Kit工具链不成熟

遇到的具体问题：

1. 规格解析不准确

   AI有时无法正确理解规格中的复杂逻辑

   需要用非常精确的语言描述，稍有歧义就可能理解错误
2. 代码生成质量不稳定

   相同的规格，不同时间生成的代码质量差异大

   有时生成的代码过于冗长，有时又过于简化
3. 增量更新困难

   规格修改后，很难做到只更新变化的部分

   往往需要重新生成整个文件，导致手工修改的部分丢失

问题3：与现有代码库集成困难

现象： 我们的代码库已经有大量历史代码，SDD更适合从零开始的新项目

- 历史代码缺乏规格说明，无法纳入SDD体系
- 新老代码风格混杂，维护成本反而增加
- 团队一部分人用SDD，一部分人用传统方式，协作困难

问题4：学习成本高

数据：

- 写出合格的第一份规格说明，平均需要3-5次迭代
- 老员工接受度较低，认为"还不如直接写代码快"

5.5 SDD适用场景分析

经过3个月的实践，我们总结出SDD的适用场景：

适合使用SDD：

- ✅ 全新的项目或模块
- ✅ 核心业务逻辑，需要长期维护
- ✅ 复杂度高，需要详细设计的功能
- ✅ 多人协作的大型需求
- ✅ 对质量要求极高的场景

不适合使用SDD：

- ❌ 简单的工具函数或配置修改
- ❌ 快速验证的实验性功能
- ❌ 一次性的临时需求
- ❌ 对现有代码的小修改

06 当前最佳实践 - Rules + Agentic Coding + AI文档汇总

6.1 融合各阶段优势

核心思路：

1. 用Rules约束AI
2. 用技术方案指导实现
3. 用Agentic Coding快速迭代
4. 用AI汇总文档保持同步

6.2 技术方案模板优化

我们优化了技术方案模板，更加轻量级：

```
# [需求名称]-技术方案
## 业务定义
[简要描述业务背景和目标，1-2句话]
## 业务领域对象
[如果需要新增/修改BO或DTO，在此说明]
## 模块领域对象
[需要新增/修改的VO对象]
| 对象含义 | 实现方案 | 属性及类型 |
|---------|---------|-----------|
| [对象名] | 新增/修改 | 1. 字段1：类型 - 说明<br>2. 字段2：类型 - 说明 |
## 数据服务层
[需要新增/修改的数据服务]
| 数据服务定义 | 实现方案 | execute逻辑 |
|------------|---------|-----------|
| [服务名] | 新增/复用 | 1. 步骤1<br>2. 步骤2 |
## 模块构建器
[需要新增/修改的模块构建器]
| 模块构建器定义 | 实现方案 | doBuild逻辑 |
|--------------|---------|-------------|
| [构建器名] | 新增/修改 | 1. 获取数据<br>2. 处理逻辑<br>3. 构建VO |
```

特点：

- 比SDD规格更轻量，编写时间从2小时降低到30分钟
- 比纯Agentic Coding更规范，有明确的结构约束
- 聚焦于"做什么"，而非"怎么做"（实现细节交给AI）

6.3 AI文档汇总机制

即：让AI自动维护"整体架构与业务逻辑文档"

6.3.1 文档汇总流程

```
完成需求开发 → 提交AI："将本次代码逻辑汇总到汇总文档" → AI分析代码 → AI更新文档
```

Prompt示例：

```
我刚完成了NN红包模块的开发，请分析以下代码：
- NnRedPacketDataService.java
```

```
- NnRedPacketModuleBuilder.java
- NnRedPacketVO.java
然后将其业务逻辑汇总到"NN业务整体架构与逻辑文档.md"中，确保：
1. 描述模块的核心功能和业务价值
2. 说明数据流转过程
3. 列出关键的业务规则和判断逻辑
4. 保持与现有文档的风格一致
```

6.3.2 架构文档结构

```
# NN业务整体架构与逻辑文档
## 一、业务概述
[业务背景、目标、核心价值]
## 二、整体架构
### 2.1 技术架构
[分层架构图、技术栈]
### 2.2 模块组成
[各个模块的功能和关系]
## 三、核心模块详解
### 3.1 NN Feeds模块
#### 3.1.1 功能说明
[模块的核心功能]
#### 3.1.2 数据流转
[数据从哪里来，经过哪些处理，最终输出什么]
#### 3.1.3 关键逻辑
[重要的业务规则、计算逻辑、判断条件]
#### 3.1.4 代码位置
[对应的类和方法]
### 3.2 NN红包模块
[类似结构]
### 3.3 NN吊钩层模块
[类似结构]
## 四、数据服务层
[各个数据服务的功能和依赖]
## 五、关键流程
[重要的业务流程时序图]
## 六、配置说明
[配置项说明]
## 七、扩展点
[未来可能的扩展方向]
```

6.3.3 AI汇总效果示例

AI生成的文档片段：

```
### 3.2 NN红包模块
#### 3.2.1 功能说明
NN红包模块用于在NN频道页面展示用户可用的红包列表，帮助用户了解可以使用的优惠，提升转
#### 3.2.2 数据流转
用户请求 → NnRedPacketDataService
          → 调用FpProvider查询用户红包
          → 过滤可用红包（状态=2，未过期，门槛<=20元）
          → 返回红包列表
          ↓
NnRedPacketModuleBuilder
          → 获取红包数据
          → 红包排序（NN专属优先，门槛从小到大）
          → 构建NnRedPacketVO
          → 返回前端
```

```
#### 3.2.3 关键逻辑
**1. 红包过滤规则**
```java
// 只展示满足以下条件的红包：
// 1. 状态为可使用（payStatus = 2）
// 2. 未过期（当前时间在startTime和endTime之间）
// 3. 门槛 <= 配置的amountThreshold（默认20元）
if (fund.getPayStatus() != 2) return false;
if (fund.getStartTime() > now || now > fund.getEndTime()) return false
if (fund.getAmountThreshold() > amountThreshold) return false;
**2. 红包排序规则**
```java
// 排序规则：
// 1. NN专属红包（配置的nnRedPacketPoolIds）优先
// 2. 其次按门槛从小到大排序
funds.stream()
    .sorted(Comparator
        .comparing(fund -> nnPoolIds.contains(fund.getFromFundId()) ?
        .thenComparing(FundQueryDTO::getAmountThreshold))
**3. 总金额计算**
```java
// 计算所有可用红包的金额总和
String totalAmount = funds.stream()
    .map(FundQueryDTO::getAvailableAmount)
    .reduce(0L, Long::sum)
    .map(PriceUtils::getPriceFenToYuan)
    .orElse("0.00");
#### 3.2.4 代码位置
```
```

```
- 数据服务：`NnRedPacketDataService`
  - 位置：`dataservice/impl/nn/NnRedPacketDataService.java`
  - 核心方法：`execute(InvocationContext, JSONObject)`
- 模块构建器：`NnRedPacketModuleBuilder`
  - 位置：`module/nn/NnRedPacketModuleBuilder.java`
  - 核心方法：`doBuild(InvocationContext)`、`sortFunds(List<FundQueryDTO
- 模块VO：`NnRedPacketVO`
  - 位置：`domain/module/nn/NnRedPacketVO.java`
#### 3.2.5 配置项
```json
{
  "nnRedPacketPoolIds": "1001,1002,1003",  // NN专属红包池ID
  "amountThreshold": 2000,                  // 红包门槛上限（分）
  "expandText": "展开查看更多"              // 展开文案
}
```



在淘特导购业务的AIcoding实践中，我们经历了从简单代码补全到Agentic Coding，再到基于规则和SDD的编程模式的演进过程。每个阶段都有其价值和局限性：

1. 初期探索让我们认识到AI在编码辅助方面的潜力，但也暴露了缺乏规范指导的问题；
2. Agentic Coding提升了功能实现的完整性，但可延续性和一致性仍有不足；
3. 基于规则的模式有效解决了代码规范和架构一致性问题，成为当前的主要实践方式；
4. SDD尝试虽然在理念上很有价值，但在实际应用中还需要进一步完善。

虽然在SDD编程方面遇到了一些挑战，但我们认为AI规范化编程是未来发展的方向。团队中的同学正在持续探索和优化：

1. 完善工具链：改进Spec Kit等工具，提升自动化能力
2. 优化流程整合：更好地将SDD模式与现有开发流程结合
3. 降低学习成本：通过培训和实践案例帮助团队成员适应新模式
4. 持续改进规则：根据实践经验不断完善规则定义

我们相信，通过持续的探索和实践，一定能找到更适合团队的AI辅助编程模式，进一步提升开发效率和代码质量。

最后

本文大部分内容由AI根据代码及配置生成。
```

08 团队介绍

本文作者式遂，来自淘天集团–淘特用户技术团队。团队主要负责淘宝行业&淘特C端链路的研发工作，包含：搜索推荐、互动游戏、导购、交易等基础服务及创新业务。当下我们积极拥抱AI时代，探索智能化在研发提效和业务场景中的无限可能。技术不只是工具，更是为用户创造价值的力量。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.3万

留言

写留言

还没有留言，来写首评